

AI Methodology Appendix — P.R.I.M.E. Documentation

ECON 3916 — ML Prediction Project · Final Submission · Spring 2026

Author: Ryan Martin · April 26, 2026

Overview

I used Claude (Anthropic) as my AI co-pilot for this project. I worked with it as one continuous conversation — not three isolated queries — so the three interactions below are representative moments from a longer back-and-forth. The prompts shown are the actual messages I sent. The work itself happened in Colab and on Streamlit Cloud, where I ran the code and verified the outputs.

P.R.I.M.E.: Prep (what I knew before asking), Request (the actual prompt), Iterate (what came back, what I changed), Mechanism Check (how I verified it), Evaluate (what I kept or rejected).

Interaction 1 — Kickoff and dataset choice

P · Prep

I had the assignment PDF and the starter notebook. I knew I wanted something real-estate related but didn't have a specific dataset in mind yet — I wanted Claude to suggest options.

R · Request (actual prompt)

"Thoroughly take your time and complete the first checkpoint (pass/fail Gate), think of/search for viable datasets to use given the recommended links he provided, i would prefer to do something real estate related, if possible, if not, totally fine, just choose something that meets all instruction requirements"

I · Iterate

Claude proposed the California Housing dataset (Pace & Barry, 1997): real-estate fit, 20,640 rows, and a known censoring quirk at \500k that would give me material for a limitations section. It drafted a checkpoint notebook covering Problem Statement, EDA, and a baseline model. The first version had things I pushed back on: a RobustScaler step before the Random Forest (pointless for tree models) and an attempt to classify the missingness as MCAR even though the data has zero missing cells. Both got removed.

M · Mechanism Check

I uploaded the notebook to Colab and ran every cell. Everything executed cleanly. I confirmed the dataset facts (N = 20,640, 8 features, target in \100k units, censored at \500,001) against the sklearn documentation page.

E · Evaluate

Kept: the dataset choice, the seven-section structure, the Tukey-fence outlier function, the geographic scatter plot. **Changed:** rewrote the missing-data section to be honest that the dataset arrived pre-cleaned. Kept all outlier rows ('legitimate low-population block groups, not data errors'). **Rejected:** the RobustScaler step, an early suggestion to log-transform the target.

Interaction 2 — Final submission build

P · Prep

Checkpoint was passed. I needed to extend it for the final rubric: at least 2 models compared, cross-validation, metrics with confidence intervals. I had Linear Regression and Random Forest from the checkpoint but no CIs and no real recommendation section.

R · Request (actual prompt)

"given what I already have completed in the checkpoint, i want you to thoroughly take your time and complete the remaining tasks of this assignment, tell me if there is anything that i need to do myself if so give me highly specific instructions."

I · Iterate

Claude added Gradient Boosting as a third model (rubric only requires 2, so this was extra) and wrote a bootstrap function for test-set CIs. It also produced the Streamlit app, requirements.txt, README, and the two PDFs. The bootstrap function:

```
def bootstrap_metric(y_true, y_pred, metric_fn, n_boot=1000, seed=42):
    rng = np.random.default_rng(seed)
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    n = len(y_true)
    boot = np.empty(n_boot)
    for i in range(n_boot):
        idx = rng.integers(0, n, size=n)
        boot[i] = metric_fn(y_true[idx], y_pred[idx])
    return boot.mean(), np.percentile(boot, [2.5, 97.5])
```

M · Mechanism Check

I sanity-checked the bootstrap on a known case: when $y_{\text{pred}} = y_{\text{true}}$ (perfect predictions), the R^2 CI should be $\sim [1.0, 1.0]$. It was. The CI widths on the actual models were tight ($\sim \pm 0.015$), which matched the 5-fold CV stability — the two methods agreeing was a good sign.

E · Evaluate

Kept: the bootstrap function, the Gradient Boosting addition, the SCR-structured Recommendation. **Asked Claude to audit** whether the deliverables actually met every rubric requirement before I submitted — useful catch on the GitHub repo step that was on me, not the AI. **Rejected:** an early plan to embed a figure built from simulated data in the report PDF. Claude flagged that this would look like real model output and was risky academically; we used tables instead.

Interaction 3 — Streamlit deployment debugging

P · Prep

I had pushed everything to GitHub and clicked Deploy on Streamlit Community Cloud. Ten minutes in, it was still building. I copied the build logs into the chat to figure out why.

R · Request (actual prompt)

"Ojay but this has taken about 10 minutes:" — followed by the Streamlit Cloud build logs showing a pillow==10.4.0 compilation failure due to missing zlib headers.

I · Iterate

Claude diagnosed it fast: Streamlit Cloud was using Python 3.14, which is too new to have pre-built wheels for pandas and pillow, so uv was trying to compile pillow from source and failing on a missing system library. The fix was two files: a runtime.txt pinning Python 3.11, and loosening requirements.txt from == to >= so pip would grab versions that had wheels.

```
runtime.txt:
    python-3.11

requirements.txt (loosened):
    streamlit>=1.31.0
```

```
pandas>=2.0.0
numpy>=1.24.0,<2.0.0
scikit-learn>=1.3.0
matplotlib>=3.7.0
seaborn>=0.12.0
joblib>=1.3.0
```

M · Mechanism Check

I committed both files and rebooted the app. Second build finished in about 3 minutes — much faster, no compile-from-source step. App loaded, all 8 sliders worked, predictions updated live, map rendered. Tested in an incognito window per the assignment's deployment guide.

E · Evaluate

Kept: both fixes. The `>=` loosening is technically less reproducible than exact pins, but the rubric just says "version pins" and exact pins kept the deployment broken. Trade-off was worth it. **Reflection:** this was probably the most useful single AI interaction of the project. Without the build log, I wouldn't have known to look at the Python version. With it, the diagnosis took about 30 seconds.

Verification & where I overrode the AI

Verification. Every code block ran in Colab or on Streamlit Cloud before being submitted. Dataset facts were confirmed against the sklearn docs. The bootstrap was sanity-checked on known cases. Empirical coverage of the 90% prediction interval came out at about 89% on the test set, which is what well-calibrated intervals should give.

Where I overrode it. A few places: (1) made the missing-data section honest about the dataset arriving pre-cleaned rather than faking an MCAR analysis; (2) kept all outlier rows because they're rural California, not data errors; (3) framed the model as a screening tool rather than a pricing tool, which made the uncertainty discussion much cleaner; (4) declined a suggestion to embed a simulated figure in the report PDF.

What the AI did well. Boilerplate. Subplot grids, the Streamlit slider scaffolding, the bootstrap function, the deployment debug. Things that would have taken hours of looking up syntax took minutes with verification.

What it got wrong. Two patterns. It kept reaching for unnecessary preprocessing (RobustScaler, log transforms) that wouldn't help tree models. And it occasionally over-engineered — class hierarchies where a few lines of NumPy would do. Both got caught at the Mechanism Check / Evaluate stage and removed.